UNDERSTANDING THE ARCHIVE · FOR THE TEAM

What GitHub Really Is

You're looking at a software-development platform — not a presentation archive. Same screen, completely different machine.

A 5-minute reframe | No jargon | For stakeholders

WHERE THIS COMES FROM

"It looks like a presentation archive — and the interface is kludgy"

That's a completely fair first reaction — and it's worth taking seriously. You open the page, you see idea folders and slide decks, wrapped in a technical-looking interface that isn't built for casual reading. So the natural conclusion is: *this is a clunky place to keep our decks.*

WHAT IT LOOKS LIKE

A folder of ideas and presentations behind a busy, developer-ish screen. A filing cabinet with too many buttons.

WHAT'S ACTUALLY GOING ON

You're judging a workshop as if it were a filing cabinet. The tool isn't clunky for storing decks — it's built for something bigger, and the decks are just what's sitting on the workbench right now.

This deck fixes the category — not to defend the interface, but to show what the interface is *for*.

FIRST, THE CORRECTION

GitHub is the world's software-development platform

Not a document store. Not an ideas app. It's the standard place where working software gets built, reviewed, versioned, and shipped — used by tens of millions of developers and effectively every serious technology company on earth. When engineers build something real, this is where it lives.

It builds software

The actual code that becomes apps and websites is written and assembled here.

It versions everything

Every change is dated, attributed, and reversible. Nothing is ever quietly overwritten.

It hands off cleanly

Any developer, anywhere, can pick up the whole project and continue it — history intact.

Everything else in this deck follows from that one fact. The ideas archive is a *use* of this machine — not the machine itself.

THE INTERFACE, HONESTLY

That "kludgy" interface is an engineering cockpit

It looks technical because it *is* a professional build environment. Every control that feels like clutter is a lever for versioning, review, or handoff. You don't have to touch any of them to read an idea — but they're the reason the archive can do things a folder of files never could.

WHAT FEELS LIKE CLUTTER	WHAT IT ACTUALLY IS
"Commits," "branches," "pull requests"	The controls for tracking every change safely
A wall of buttons and tabs	A cockpit for building and reviewing work
Timestamps and version labels everywhere	The guarantee that nothing is ever lost
Technical file names and folders	The structure that lets anyone find their way in

Judging it by the interface is like judging a recording studio's mixing board by whether it's a comfy place to read a book. Wrong test for the machine.

THE REFRAME

Stop reading it as a document tool

Same screen, two completely different mental models. Swap the one on the left for the one on the right and the whole thing starts making sense.

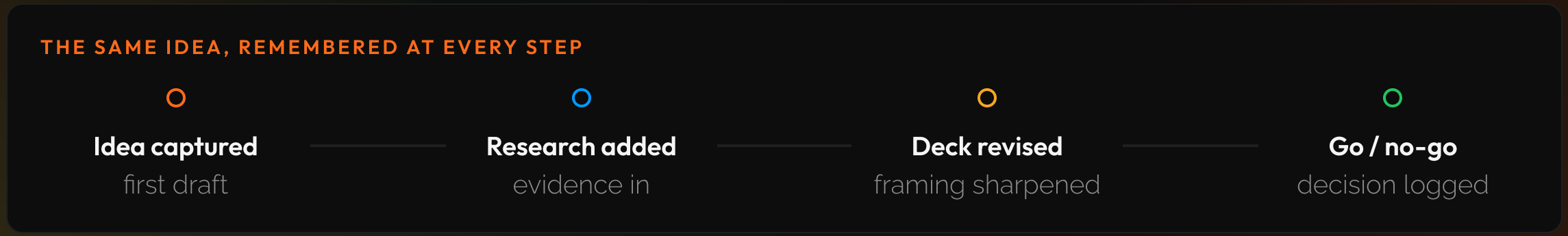
✕ A presentation archive	✓ A versioned platform we build in	The decks are output, not the point.
✕ A kludgy interface	✓ Professional engineering controls	Built for precision, not for browsing.
✕ Storage for finished files	✓ A living record with memory	It remembers every step, not just the last one.
✕ A folder on someone's computer	✓ A project anyone can continue	Not trapped in one laptop or one head.

The rest of this deck is just the three things that right-hand column buys us.

SUPERPOWER 1

Nothing is ever lost — the entire history is kept

Think of Google Docs version history — but for an entire project, not one file. Every change is a dated snapshot: who made it, when, and what it replaced. You can rewind to any point, compare any two moments, and recover anything. There is no "final_v3_REALLY_final" problem here.



Every one of those dots is still there, forever. You can always see how we got here.

SUPERPOWER 2

It keeps the *why*, not just the files

A shared drive stores *what* a document says. This stores the reasoning around it — the discussion, the objections, the decision, and who was in the room. When you come back in a year and ask "why did we do it this way?", the answer is right there, attached to the work.

A PLAIN SHARED DRIVE

The latest file, and nothing else

No record of what was debated

"Ask whoever made it" — if they're still here

Decisions live in email threads that get lost

THIS ARCHIVE

The discussion is pinned to the work itself

Every decision has a paper trail

Who changed what, when, and why

The full story survives the people who wrote it

This is called provenance — a complete, trustworthy record of how a piece of work came to be. It's the difference between a filing cabinet and an institutional memory.

SUPERPOWER 3

Anyone can pick it up and continue the line

This is the one that matters most for us. Because the whole project lives here — with its full history — any developer, anywhere, can take a complete copy and keep building. In GitHub's language that's called *forking*: photocopy the entire lab notebook, every past page intact, and continue it as your own line.

NO LOCK-IN

The work isn't trapped in one vendor, one tool, or one person's laptop. It's portable by design.

NO SINGLE POINT OF FAILURE

If the person who built it moves on, the next developer starts from exactly here — not from scratch.

NATIVE HABITAT FOR DEVELOPERS

Fork, copy, review, hand off — this is exactly how software teams collaborate worldwide, every day.

"Any developer can pick up from here" is literal. That portability is the whole reason the work is safe.

HOW WE ACTUALLY USE IT

We stage ideas in the same machine they'll be built in

Here's the deliberate part. We could keep early ideas in a doc somewhere and only "move to GitHub" once they become code. We don't — on purpose. We incubate ideas *here* so that the moment one graduates into real software, it's already home: same platform, same history, nothing to migrate.

WHAT LIVES HERE TODAY

Numbered idea folders — one per idea

Discussion papers — the written case

Decks — the pitch, like this one

Research — the evidence behind it

Issues — the running debate

WHY PUT IT HERE FIRST

Every idea arrives already versioned, already discussable, already portable. When Scott greenlights one, it doesn't get "set up" — it just graduates into its own project, carrying its entire history with it. No handoff gap.

We're not storing ideas in a coding tool by accident. We're staging them in the front door of the factory where they'll be built.

ANATOMY OF ONE IDEA

What's actually inside a single idea folder

The "technical clutter" is simpler than it looks. Open any idea and here's the whole map — five things, plain-English names:

`README.md`	The discussion paper	The written case for the idea — what opens automatically when you click the folder.
`presentations/`	The pitch	The source for decks like this one.
`research/`	The proof	Market research, sources, and evidence behind the claims.
`exports/`	The shareable version	The finished deck as a PDF or a full-screen web link.
Issues & history	The debate	The running conversation and every change, dated and attributed.

That's it. README is the paper; the rest is supporting material. Once you know the map, the "clutter" disappears.

THE PAYOFF OF STAGING IT HERE

Idea → its own repo → built as software, history intact

The reason the front-of-house looks like a developer's tool is that it *is* the on-ramp to one. An idea never has to leave the platform to become real.

Captured
idea gets a folder

→

Researched
evidence + debate

→

Go / no-go
decision logged

→

Graduates & is built
own repo — history carried in

At no point does anyone re-key the work into a "real" system. The idea and the software it becomes are the same continuous thread — which is exactly why we start it here.

WHY THIS MATTERS FOR THE ORGANIZATION

Continuity, ownership, and no single point of failure

This isn't a developer convenience. For Arterial, Not Real Art, and every brand, it's an operational safeguard.

Institutional memory

The reasoning behind every idea and decision survives staff changes. It doesn't walk out the door.

You own the work

It's portable and vendor-neutral. No platform can hold your ideas or code hostage.

Any developer can continue

Not dependent on one person being available. The next builder starts from exactly here.

The "kludgy tool" is the thing making sure the work outlasts whoever happened to make it.

WAYFINDING

How to read the repo in 60 seconds

You never need to touch a single developer control. Here's the whole path — ignore everything else on the screen.

1. **Start at the front-page table.** The main README lists every idea with a one-line status. That's your index.
2. **Click an idea's folder.** The discussion paper opens automatically underneath it — that's the full written case.
3. **Click the deck thumbnail near the top.** It opens a full-screen, arrow-key presentation — no PDF tab, no download.

— **Everything else is plumbing.** Buttons, tabs, and technical files are for the people building. You can safely ignore them.

Table → folder → deck. That's the whole tour.

It's not a presentation archive

It's the workshop where the work gets built —
and survives the people who made it.

Same screen, completely different machine.